

MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent

Hongli Yu^{1,2,3}, Tinghong Chen², Jiangtao Feng², Jiangjie Chen^{1,3}, Weinan Dai^{1,2,3},
Qiyang Yu^{1,2,3}, Ya-Qin Zhang^{2,3}, Wei-Ying Ma^{2,3}, Jingjing Liu^{2,3}, Mingxuan Wang^{1,3}, Hao Zhou^{2,3}

¹ByteDance Seed ²Institute for AI Industry Research (AIR), Tsinghua University

³SIA-Lab of Tsinghua AIR and ByteDance Seed

Abstract

Despite improvements by length extrapolation, efficient attention and memory modules, handling infinitely long documents with linear complexity without performance degradation during extrapolation remains the ultimate challenge in long-text processing. We directly optimize for long-text tasks in an end-to-end fashion and introduce a novel agent workflow, MEMAGENT, which reads text in segments and updates the memory using an overwrite strategy. We extend the DAPO algorithm to facilitate training via independent-context multi-conversation generation. MEMAGENT has demonstrated superb long-context capabilities, being able to extrapolate from an 8K context trained on 32K text to a 3.5M QA task with performance loss < 5% and achieves 95%+ in 512K RULER test.

Date: June 26, 2025

Correspondence: zhouhao@air.tsinghua.edu.cn, wangmingxuan.89@bytedance.com

Project Page: <https://memagent-sialab.github.io/>

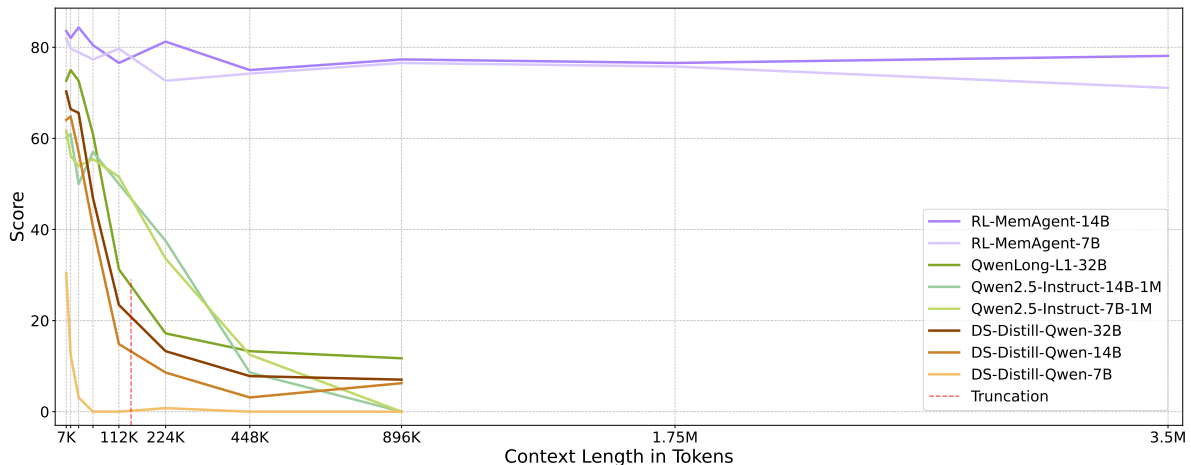


Figure 1 Accuracy scores of RULER-HotpotQA [1, 2]. Even models that employ long-context continual pretraining and extrapolation techniques fail to maintain consistent performance. In contrast, MEMAGENT with RL demonstrates nearly lossless performance extrapolation.

1 Introduction

While having demonstrated impressive capabilities [3–7], industry-level Large Language Model (LLM) systems [8–10] still face a critical challenge: how to handle long contexts effectively - processing an entire book, executing a complex chain of reasoning over many steps, or managing the long-term memory of an agent system - all these complex tasks can generate overflowing text that quickly explodes the typical-size context window of current LLMs.

Existing approaches to long-context tasks are three-pronged. The first involves length extrapolation methods by shifting the positional embeddings in order to extend the context window of the model [11–15], plus continued pre-training [16–18]. Despite promising potential, these methods often suffer from performance degradation and slow processing speed due to $O(n^2)$ computational complexity when applied to extremely long text. The second school of methods leverages sparse attention [19–21] and linear attention mechanisms [22, 23] to reduce the complexity of attention for more efficient processing of longer sequences. However, this typically requires training from scratch, with inherent adversities such as linear attention facing difficulties in parallel training or sparse attention depending on human-defined patterns. The last line of inquiry investigates context compression [24–27], which aims to condense information in token-level or external-memory-plugin modules. Such approaches often struggle with extrapolation, and require the integration of additional modules or context operations, which ineluctably disrupts the standard generation process and hinders compatibility as well as parallelization.

Hence, a successful LLM with strong long-context capabilities requires the trinity of: 1) processing infinite length of text; 2) scaling without performance drop; and 3) efficient decoding with linear complexity. To pursue this quest, we return to the basic intuition behind long-context modeling [28–31]. When humans process long-context information, we tend to abstract out the main revealing conceptions to capture the essence of the whole text, often by making notes of critical details or using short-handed stenograph to record the key points, while discarding redundant and irrelevant data. We do not attempt to memorize every single fact or each small piece of information; instead, we focus our intellectual energy on more important aspects of the task at hand. This selective attention not only simplifies the process but also aids in tackling complex problems more efficiently.

Following this anthropocentric intuition, we propose a novel use of Reinforcement Learning (RL) to equip LLMs with a dynamically updated fixed-length ‘memory’, as illustrated in Figure 2. During inference, the LLM processes the input text segment-by-segment. As it reads each segment, the model proactively and selectively updates the memory, which then contributes to the generation of the final output after all relevant messages are aggregated and synergized in the memory. This clever mechanism allows the LLM to flexibly handle arbitrary text lengths while maintaining a linear time complexity during processing, since the length of the memory is fixed, which leads to a fixed context window size for the model. This segment-based approach generates multiple outputs from a single long-text input, requiring multiple rounds of memory updates and a final round for the generation of the final response. Training this type of agent workflow, which enables dialogues across multiple independent contexts, is still an unexplored territory in current LLM study. Existing systems typically handle workflow trajectories via alternating tool calls or environment feedback by either simply concatenating [32, 33] them or using a sliding window [34] approach, which lacks flexibility and scalability in practice. Our MEMAGENT approach, instead, proposes that treats each context-independent conversation as an optimization objective. Based on the DAPO[35] algorithm, we implement the Multi-Conv DAPO to optimize an arbitrary agent workflow by verifiable outcome reward.

In our experiments, an RL-trained model with a modest 8K context window (with a 1024-token memory and a 5000-token document chunk) trained on 32K documents exhibits consistently superb capabilities for Question Answering (QA) tasks on documents of up to 4 million tokens, without performance drop and with linear computation cost. This demonstratively showcases the efficiency and scalability of our long-context memory approach.

Our major contributions are threefold:

- We introduce a novel approach that enables LLMs to process arbitrarily long inputs within limited

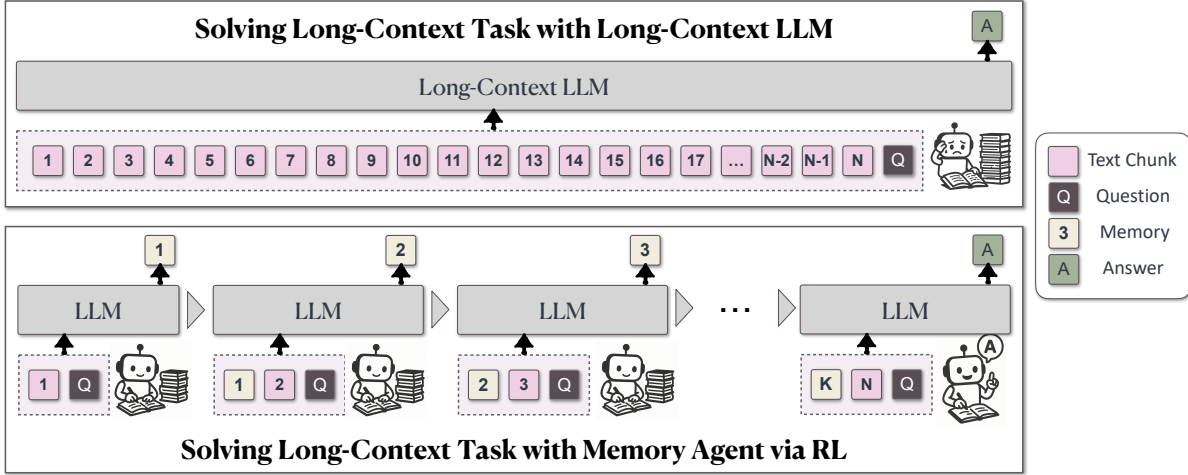


Figure 2 MEMAGENT is inspired by the way humans process long documents. It divides the document into multiple chunks and allows LLMs to process them iteratively, recording relevant information in memory. Finally, LLMs generate answers based on the information stored in the memory.

context window under linear time complexity during inference, overcoming a significant bottleneck in long-context processing.

- We design an agent workflow to implement this mechanism and propose an end-to-end training approach using the multi-conversation DAPO algorithm.
- We empirically demonstrate that our RL-trained method allows models to extrapolate to vastly long documents with minimal performance degradation, pushing the boundaries of what is currently achievable in long-context LLM systems.

2 Related Work

Long Context LLMs Extrapolation methods for RoPE-based LLMs [11], such as NTK [12], PI [13], YaRN [14] and DCA [15], modify the base frequency, position index and other components of positional embeddings, enabling the model to capture long-range semantic dependencies. On the other hand, Linear attention mechanisms [22, 23], Recurrent Neural Networks (RNNs) and State Space Models (SSMs) [36–40] leverage different architectures to achieve $O(N)$ computation complexity, aiming to process extremely long context. Sparse attention [19–21] shifts the attention mask matrix to apply patterns such as sliding windows, thereby eliminating irrelevant attention calculations. However, these patterns are typically based on predefined heuristics. The possibility of dynamic sparse attention [41, 42] has been explored recently. The Long Short-Term Memory (LSTM) mechanism [29] achieved significant success in early NLP tasks, while Neural Turing Machines [30] and Memory Networks [31] demonstrated how to equip neural networks with memory. Existing memory mechanisms integrated to Transformer models are typically realized by adding external memory modules [26, 43–45] or external database [46–48]. In contrast, we use reinforcement learning (RL) to enable LLM itself the ability to memorize.

Reinforcement Learning for LLMs In recent RL studies, the reward signals have gradually shifted from human preferences [49] or reward models distilled from them [50] to rule-based feedback, which has demonstrated great potential in enhancing model reasoning capabilities [3, 4, 51–53]. Key contributions include PPO [54] based on GAE [55], the Actor-Critic framework, as well as GRPO [56] that utilizes Group Normalization. Algorithmic enhancements [35, 57, 58] have mostly focused on improving training sustainability and sample efficiency of these algorithms. To further release the potential of RL, recent works such as Search-R1 [33], Agent-R1 [32] and RAGEN [59] have explored the training of tool-using agents based on multi-turn chat. However, these multi-turn chats are constructed by alternately concatenating tool responses and model

replies, with the ultimate optimization goal being a single conversation with tool masking. GiGPO [34] further investigates the use of multiple independent contexts in agent training with environment feedback with sliding window trajectories. However, these approaches are limited to optimizing interleaved trajectories of observation and generation, making them difficult to apply to more general agent workflows.

3 The Proposed MemAgent

In this section, we describe the details of MEMAGENT approach for solving long-context tasks, including the overall workflow (§ 3.1), Multi-conv RL algorithm for training MEMAGENT (§ A), reward modeling design (§ 3.3), and architecture implementation design (§ 3.4).

3.1 The MemAgent Workflow: RL-shaped Memory for Unbounded Contexts

As illustrated in Figure 2, MEMAGENT views an arbitrarily long document not as a monolithic block but as a controlled *stream* of evidence. At every step, the model sees exactly two things: the next chunk of text and a compact, fixed-length *memory* that summarizes everything deemed important so far. Crucially, the memory is just a sequence of ordinary tokens inside the context window, so the core generation process of the base LLM remains unchanged.

After reading a new chunk, the model overwrites the previous memory with an updated one. This **overwrite** strategy seems almost too simple, yet it is precisely what enables the system to scale: because memory length never grows, the total compute per chunk stays $O(1)$ and end-to-end complexity is strictly linear to the number of chunks. We formulate the overwrite decision as a reinforcement learning problem: the agent is rewarded for retaining information that will later prove useful and for discarding distractors that would waste precious tokens. By optimizing this objective with our newly introduced multi-conversation DAPO algorithm (detailed in § A), the model learns to compress aggressively while preserving answer-critical facts.

The workflow naturally decomposes inference into two modules. Within the **Context-Processing** module the model iterates over chunks, updating memory with a prompt template (Table 1, top). Once the stream is exhausted, a final **Answer-Generation** module is invoked (Table 1, bottom) where the model consults only the problem statement and the memory to produce its boxed answer. Because positional embeddings are never re-scaled or patched, the same tokenization and attention layout apply in both modules, unlocking the model’s latent length-extrapolation capability without any architectural modifications.

MEMAGENT therefore enjoys three benefits from this design: (1) **Unlimited length**: the document can be millions of tokens because it is processed as a stream; (2) **No performance cliff**: RL encourages the memory to retain exactly the information needed, yielding near-lossless extrapolation (Figure 1); (3) **Linear cost**: a constant window size implies decoding time and memory consumption grow linearly with input length ($O(N)$) (detailed in § A.) This renders a practical recipe for turning any moderately context-sized LLM into an efficient long-context reasoner with minimal engineering overhead.

3.2 Training MemAgent with Multi-conv RL

By viewing memory update in context processing for answer-generation tasks as part of the policy to be optimized by RL, we adopt the RLVR recipe [3, 51, 60] to train MEMAGENT.

For the base algorithm, we adopt Group Relative Policy Optimization (GRPO) [56] for its simplicity and effectiveness in RLVR. In the rollout phase of GRPO, the policy model $\pi_{\theta_{\text{old}}}$ samples a group of G individual responses $\{o_i\}_{i=1}^G$ for an input x . Let $\{R_i\}_{i=1}^G$ refer to the sequence-level rewards, then the group normalizing advantage of the i -th response is calculated by the following function:

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (1)$$

You are presented with a **problem**, a **section** of an article that may contain the answer, and a **previous memory**. Please read the section carefully and update the memory with new information that helps to answer the problem, while retaining all relevant details from the previous memory.

<problem> {prompt} </problem>

<memory> {memory} </memory>

<section> {chunk} </section>

Updated memory:

You are presented with a **problem** and a **previous memory**. Please answer the problem based on the previous memory and put the answer in \boxed {}.

<problem> {prompt} </problem>

<memory> {memory} </memory>

Your answer:

Table 1 Template of MEMAGENT for context processing (top part) and final answer generation (bottom). Curly-brace placeholders {} will be replaced with actual content.

GRPO adopts a clipped objective with a KL penalty term:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right) - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right], \quad (2)$$

where $r_{i,t}(\theta)$ refers to the importance sampling weight:

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}. \quad (3)$$

However, due to the nature of the MEMAGENT approach, it generates multiple context-independent conversations for a single query, as illustrated in Figure 2. Therefore, policy optimization cannot be implemented by simply applying the attention mask as is done in multi-turn tool-calling optimization.

To address this issue, we treat each conversation as an independent optimization target, as demonstrated in Figure 3. Let n_i denote the number of generated conversations $(o_{i,1}, o_{i,2}, \dots, o_{i,n_i})$ for a given sample (q_i, a_i) in a group. $o_{i,j}$ further decomposes into token-level outputs $(o_{i,j,1}, o_{i,j,2}, \dots, o_{i,j,|o_{i,j}|})$. We compute an outcome reward R_i per sample by the final conversation that contains the final answer, and distribute group-normalized advantages across all associated conversations.

Equations 4 and 5 illustrate how the advantage and loss are computed within our MEMAGENT algorithm for context-independent multi-conversation rollouts. The advantage value is derived from the conversation that contains the final answer, then uniformly applied across all conversations originating from the same sample. Our loss function is analogous to that used in DAPO [35], which incorporates a token-level averaging loss. Furthermore, we extend the dimensionality of the loss computation from the conventional (**group**, **token**) structure to (**group**, **conversation**, **token**). Following DrGRPO [58], we do not normalize the advantage by the standard deviation of rewards.

$$\hat{A}_{i,j,t} = r_i - \text{mean}(\{R_i\}_{i=1}^G) \quad (4)$$

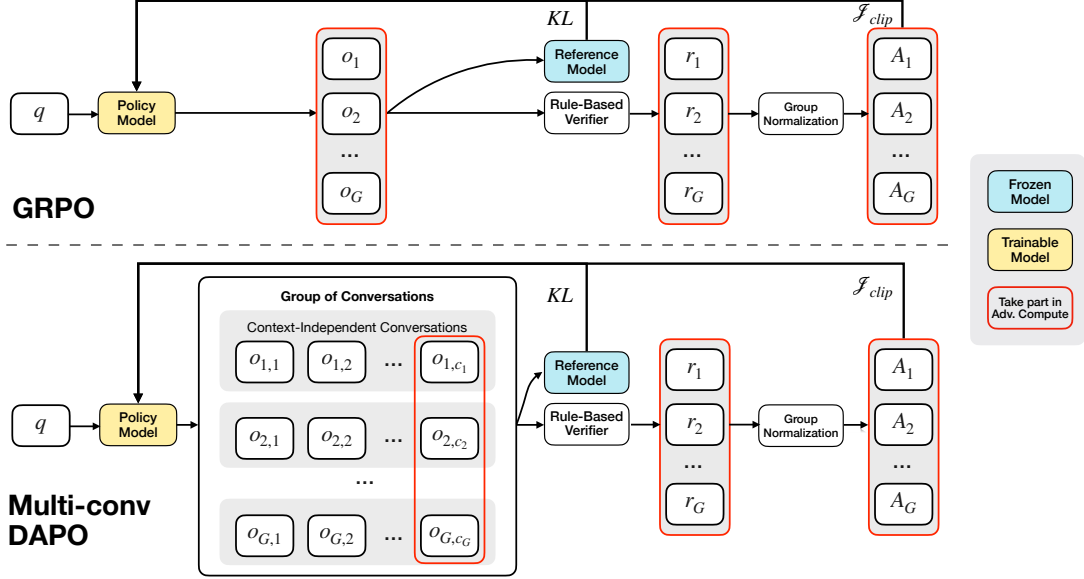


Figure 3 Comparison between vanilla GRPO and Multi-Conv DAPO. During the rollout phase of Multi-conv DAPO, each sample generates multiple conversations. The answer contained in the final conversation is used to compute the reward and advantage, which are then employed to optimize all preceding conversations.

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_{i,j}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q, o_{i,j-1})} \left[\frac{1}{\sum_{i=1}^G \sum_{j=1}^{n_i} |o_{i,j}|} \sum_{i=1}^G \sum_{j=1}^{n_i} \sum_{t=1}^{|o_{i,j}|} \left(\mathcal{C}_{i,j,t} - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right] \quad (5)$$

$$\text{where } \mathcal{C}_{i,j,t} = \min \left(r_{i,j,t}(\theta) \hat{A}_{i,j,t}, \text{clip} \left(r_{i,j,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}} \right) \hat{A}_{i,j,t} \right)$$

3.3 Reward Modeling

Following the RLVR recipe [33, 35, 51], we train the model with a final outcome reward computed by a rule-based verifier. In RULER [1] and other datasets, questions may have multiple ground-truth answers. For some tasks, such as question answering, these ground truths are considered equivalent. Given a set of multiple ground-truth answers $Y = \{y_1, y_2, \dots, y_n\}$, the reward score is defined as:

$$R(\hat{y}, Y) = \max_{y \in Y} (\mathbb{I}(\text{is_equiv}(y, \hat{y}))) \quad (6)$$

where $\hat{y}$ is the predicted answer, and $\mathbb{I}(\cdot)$ denotes the indicator function.

For other tasks, all ground-truth answers are expected to be included in the final output. An example is the task of Multi-Value Needle in a Haystack, where the question might be: “What are all the special magic numbers for XXX?” In such cases, the reward function is defined as:

$$R(\hat{y}, Y) = \frac{|y \in Y \mid \mathbb{I}(y \in \hat{y})|}{|Y|} \quad (7)$$

where $|\cdot|$ denotes the cardinality of a set.

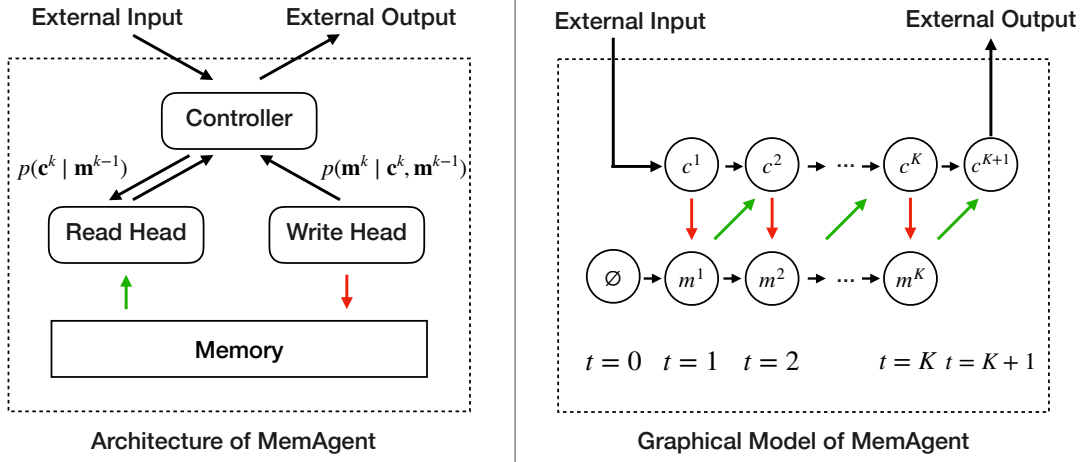


Figure 4 The architecture and graphic model of MEMAGENT. The memory is modeled as a latent memory variable, thereby enabling the decomposition of the autoregressive language model into multiple steps of reading from and writing to the memory.

3.4 Rethinking MemAgent from Autoregressive Modeling Perspectives

Finally, to get a deeper sense of the MEMAGENT design, we propose to re-think language-model factorization in the following fashion. A standard autoregressive LLM factorizes the joint likelihood of a sequence $\mathbf{x}_{1:N}$ as $p(\mathbf{x}_{1:N}) = \prod_{n=1}^N p(x_n | \mathbf{x}_{1:n-1})$, implicitly assuming that every past token (or at least its hidden state) must stay in the active context. This is what turns quadratic attention into the long-context bottleneck.

MEMAGENT replaces the unbounded history with a fixed-length *memory* $\mathbf{m} \in \mathbb{V}^M$, as shown in Figure 4. The input text is streamed through the model in K contiguous chunks $\mathbf{c}^1, \dots, \mathbf{c}^K$ (each of length $\leq C$). After chunk k is read, the model overwrites the panel with a new vector $\mathbf{m}^k$ that summarizes *all* evidence seen so far. Because $|\mathbf{m}^k| = M$ is constant, both compute and memory per step are $O(C + M)$, yielding an overall linear complexity $O(N)$.

Introducing the latent sequence $\mathbf{m}^{1:K-1}$ decomposes the original likelihood as

$$p(\mathbf{x}_{1:N}) = \sum_{\mathbf{m}^{1:K-1}} \prod_{k=1}^K \underbrace{p(\mathbf{c}^k | \mathbf{m}^{k-1})}_{\text{read}} \underbrace{p(\mathbf{m}^k | \mathbf{c}^k, \mathbf{m}^{k-1})}_{\text{write}}, \quad (8)$$

with base case $\mathbf{m}^0 = \emptyset$. Inside each chunk, we still run an ordinary transformer decoder, but conditioned on a *constant* context window $(\mathbf{c}^k, \mathbf{m}^{k-1})$. The read path factorizes token-by-token, $p(\mathbf{c}^k | \mathbf{m}^{k-1}) = \prod_{i=(k-1)C+1}^{kC} p(x_i | \mathbf{x}_{1:i-1}, \mathbf{m}^{k-1})$, while the write path generates the next memory in the same autoregressive fashion.

MEMAGENT enjoys token-level compression of context, yet local-global or linear-attention models compress long context in the *feature* space; their summaries are implicit and opaque. MEMAGENT’s summaries reside in token space, so every intermediate memory is human-readable and can be inspected or even edited — a property we exploit when designing the RL reward (§3.3). Conceptually, Equation 8 turns the transformer into a recurrent network whose state size is user-controllable.

Why is RL Essential? Because memory tokens are latent and updated via a discrete overwrite rule, back-

propagation alone cannot teach the model *what* to keep and what to discard. Our multi-conversation GRPO algorithm (§A) treats each read–write–read loop as an RL transition, directly rewarding memories that lead to a correct final answer. This bridges the gap between explicit supervision (answers) and implicit structure (good memories), completing the training pipeline introduced earlier.

The resulting MEMAGENT architecture preserves the vanilla decoder’s training recipe, requires no exotic attention kernels, and satisfies the long-context trilemma of arbitrary length, lossless extrapolation, and linear cost.

4 Experiments

For our training and primary evaluation, we utilize multi-hop long-text question answering (QA) tasks, and further conduct evaluations on other various long-text tasks. We select prior long-context methods as baselines to evaluate the long-text extrapolation capabilities of the models by comparing performance changes as the length of the test set data increases.

4.1 Datasets

RULER [1] comprises various synthetic tasks with controllable context lengths, making it an ideal benchmark for investigating how model performance varies with increasing context length.

The Question Answering subset of RULER adapts existing short-context QA datasets for long-context evaluation by embedding golden paragraphs (containing correct answers) within extensive distractor content sampled from the same dataset. This configuration represents a real-world adaptation of the Needle in a Haystack (NIAH) paradigm, where questions serve as queries, golden paragraphs function as needles, and distractor paragraphs constitute the haystack. This task bridges the gap between synthetic evaluation and practical long-context applications, well poised for assessing a model’s ability to locate and extract relevant information from realistic document collections.

We synthesized training samples from the HotpotQA dataset using this methodology. Our synthetic data comprises a total of 200 articles, with an approximate token length of 28K.

We thoroughly cleaned our dataset by filtering out questions where the Best-Of-2 score is 100% without requiring any context for Qwen2.5-7B-Base or Qwen2.5-7B-Instruct. These questions likely represent common knowledge already internalized within the models’ memories. Using this method, we processed 80,000 samples from the HotpotQA [2] training split. Approximately 50% of the data were filtered out, and from the remaining samples we selected the first 32,768 samples for further use.

We then applied a similar approach to synthesize 128 samples from the HotpotQA validation set. To further investigate how model performance varies with length, we synthesized test sets with different context lengths using the same questions. The number of articles ranges from 50, 100, up to 6400, corresponding to context lengths of approximately 7K, 14K, and up to 3.5M tokens, respectively.

4.2 Experimental Setup

Training Details To maintain comparability with previous work, we choose Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct as base models for experiments. We implement the framework for multi-conversation with independent contexts based on verl [61]. During training, we intentionally limit the model to an 8K context window to highlight its extrapolation capabilities. This 8K-window was allocated as follows: 1024 tokens for the query, 5000 tokens for the context chunk, 1024 tokens for the memory, and 1024 tokens for the output, with remaining tokens reserved for the chat template. Consequently, the model typically requires 5 to 7 conversational turns to process the entire context.

Hyperparameters We use the GRPO algorithm for training, applying a KL factor of 1e-3 and disabling the entropy loss. We employ the AdamW optimizer with a learning rate of 1e-6, scheduled with a constant learning

rate with linear warm-up. We use a rollout batch size of 128 and 256 for 7B and 14B models, respectively, and a group size of 16. The ratio of the sample batch size to the backpropagation batch size is set to 16.

Model Configuration We use DeepSeek-R1-Distill-Qwen [51], Qwen2.5-Instruct-1M [62] and QwenLong-L1 [63] as baselines. We follow the official configurations of these baseline models to set context lengths. Specifically, for the Qwen2.5-Instruct-1M series, we further extrapolate the context length to 1M tokens using DCA. For the DeepSeek-R1-Distill-Qwen series and QwenLong, the context length is set to 128K tokens. For the model with 128K context length, the input consists of 120,000 tokens, with an output of 10,000 tokens. For the model with 1M context length, the input is 990,000 tokens, with an output of 10,000 tokens.

4.3 Main Results

The main experimental results are reported in Table 2. We conduct a comparative analysis of all model performances within the context length ranging from 7K to 896K. Specifically, for the MEMAGENT model, we extend our evaluation to explore its extrapolation capabilities on ultra-long contexts of 1.75M and 3.5M, assessing how the model generalizes beyond the standard context range.

From these results, we observe that MEMAGENT exhibits remarkable length extrapolation capabilities with only marginal performance decay as the input context length increases. This demonstrates the effectiveness of the proposed memory mechanism combined with reinforcement learning for handling ultra-long context scenarios.

In contrast, baseline models demonstrate distinct failure patterns even within the context window. The reasoning models (DS-Distill-Qwen series) show rapid performance degradation, while QwenLong-L1 maintains reasonable performance within its training length 60K but experiences substantial degradation afterward. The Qwen2.5-Instruct-1M series models maintains an acceptable performance within 112K tokens. However, their performances deteriorate to zero at 896K tokens, well before reaching their theoretical 1M token capacity. This suggests that despite extended context windows, these models struggle with effective information utilization in ultra-long contexts.

Table 2 Main experimental results comparing model performance across various context lengths. All values represent accuracy (%).

Model	Length									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
QwenLong-L1-32B	72.66	75.00	72.66	60.94	31.25	17.19	13.28	11.72	N/A	N/A
Qwen2.5-Instruct-14B-1M	60.16	60.94	50.00	57.03	50.00	37.50	8.59	0.00	N/A	N/A
Qwen2.5-Instruct-7B-1M	61.72	56.25	53.91	55.47	51.56	33.59	12.50	0.00	N/A	N/A
DS-Distill-Qwen-32B	70.31	66.41	65.62	46.88	23.44	13.28	7.81	7.03	N/A	N/A
DS-Distill-Qwen-14B	64.06	64.84	57.03	40.62	14.84	8.59	3.12	6.25	N/A	N/A
DS-Distill-Qwen-7B	30.47	12.50	3.12	0.00	0.00	0.78	0.00	0.00	N/A	N/A
RL-MEMAGENT-14B	83.59	82.03	84.38	80.47	76.56	81.25	75.00	77.34	76.56	78.12
RL-MEMAGENT-7B	82.03	79.69	78.91	77.34	79.69	72.66	74.22	76.56	75.78	71.09

4.4 Ablation Study

RL Training To investigate the impact of reinforcement learning on the memory mechanism, we conduct further ablation experiments. Our baselines are Qwen2.5-Instruct [64] series and Qwen2.5-Instruct models which are equipped with memory mechanism without RL training.

As shown in Figure 5, vanilla models exhibit severe performance degradation as context length increases, especially after 112K where the inputs are truncated because of the context window. While the model equipped with a memory, without RL training, demonstrates better performance and maintains reasonable

performance on tasks exceeding the context length, it still experiences an overall decline in performance as the input length increases.

In contrast, RL-trained models maintain consistently high performance across all context lengths with minimal degradation. This demonstrates that while the memory mechanism provides structural support for long contexts, reinforcement learning is essential for teaching models to properly leverage the memory.

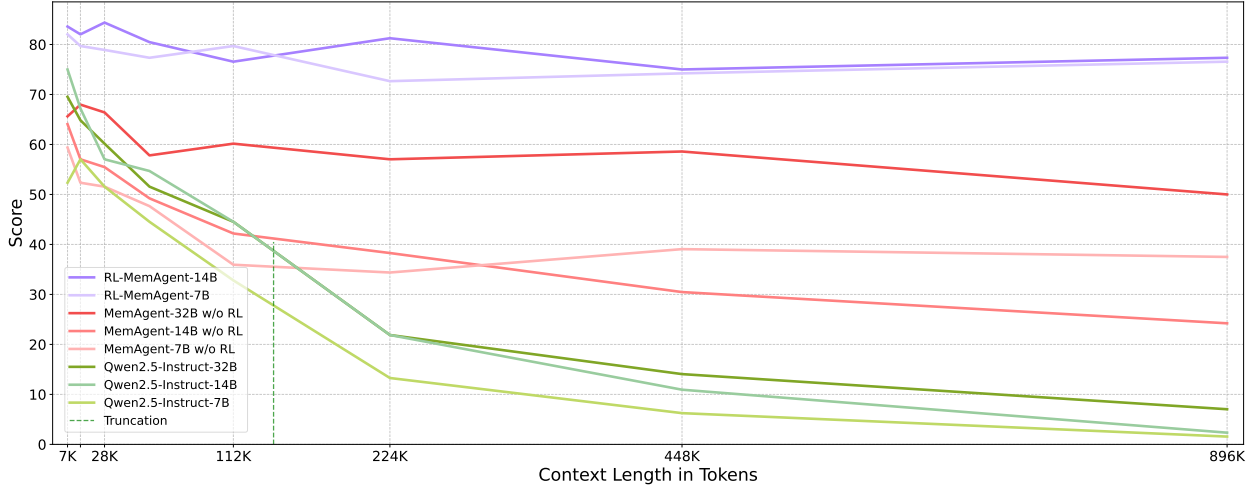


Figure 5 Ablation study on RULER-HotpotQA comparing models with and without RL training across context lengths from 28K to 896K tokens.

Out-of-Distribution Tasks To evaluate the generalization capabilities of our approach, we conduct comprehensive experiments on the OOD task in RULER benchmark, including **needle-in-a-haystack variants**, **variable tracking**, **frequent words extraction**, and **question answering** synthesized from SQuAD [65]. We synthesize context lengths ranging from 8K to 512K tokens for these tasks, except that SQuAD extends only to 256K tokens due to limited document length.

Figure 6 presents the performance comparison across different task categories. The results demonstrate that MEMAGENT maintains consistently superior performance across diverse task types. Particularly, MEMAGENT-14B achieves over 95% accuracy on the average RULER tasks in context ranging from 8K to 512K, while MEMAGENT-7B achieves the best performance, surpassing 32B model without RL training and long-context post-trained models. MEMAGENT-7B/14B both maintain stable performance on the SQuAD-based QA task, indicating that memorizing ability can generalize beyond training data. In contrast, baseline models show significant degradation beyond 128K tokens across all task categories.

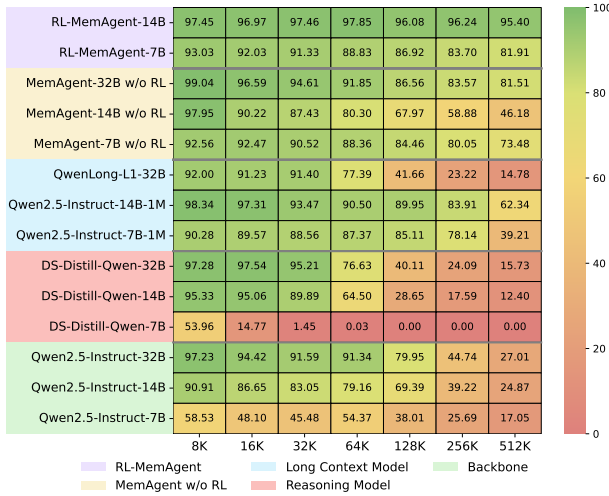
The consistent performance strength across heterogeneous tasks validates that the memory mechanism effectively generalizes to various long-context scenarios rather than overfitting to specific formats. Complete results for all individual RULER tasks are provided in Appendix B.

4.5 Case Study

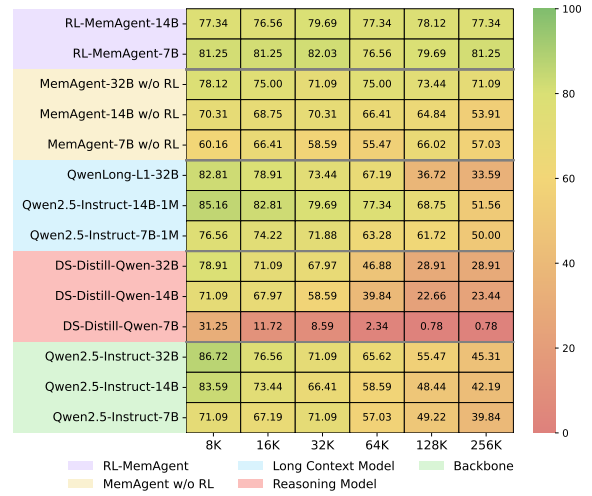
To further illustrate the proposed memory mechanism in detail, we conduct a case study on a generation trajectory of MEMAGENT-14B. The input question is: *The director of the romantic comedy ‘Big Stone Gap’ is based in what New York city?* This is a 2-hop question with the following relevant Wikipedia entries:

- 1) **Big Stone Gap** is a 2014 American drama romantic comedy film written and directed by Adriana Trigiani.
- 2) **Adriana Trigiani** is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in Greenwich Village, New York City.

In the first round, the model is presented with the entry *Ghost*, which refers to a production team also based in *New York*. The model chooses to retain this potentially useful information for future use. In the second



(a) RULER average across 10 tasks



(b) RULER-QA task from SQuAD

Figure 6 Performance heatmaps on RULER benchmark tasks showing accuracy scores across different context lengths (greener indicates better performance). Models are grouped by type on the vertical axis. (a) Average performance across 10 synthetic tasks including needle-in-a-haystack variants, variable tracking, and word extraction. (b) Question answering task synthesized from SQuAD dataset with context lengths up to 256K tokens.

round, no relevant context is provided; nevertheless, the model maintains its agent state, demonstrating robustness against distraction. In the third round, both relevant entries are presented. The model correctly identifies critical information and updates its memory accordingly, leading to the correct answer: *Greenwich Village, New York City*. At this point, the reasoning process is complete. In the remaining rounds, the model’s memory remains unchanged and is used to produce the final response.

Chunk1

...

Document 20: Ghost (production team)

Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in *New York City*.

...

Memory1

The provided section does not contain any information about the director of the romantic comedy "Big Stone Gap" or their base in New York City. **However, it does mention a production team called Ghost which is based in New York City.** Since the problem is specifically asking about the director of "Big Stone Gap," and there is no direct information about the director in the given section, we cannot infer anything directly related to the problem from the provided text.

Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn’s three first studio albums.

The problem remains unanswered based on the provided section. Additional information would be needed to determine the director of "Big Stone Gap" and their base in New York City.

Chunk2

...(No relevant document)

Memory2

The provided section does not contain any information about the director of the romantic comedy **Big Stone Gap** or their base in New York City. Therefore, the updated memory remains unchanged with respect to the problem at hand. The memory still notes that the Ghost record producing and songwriting team is based in New York City, but it does not provide any information about the director of "Big Stone Gap."

Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn's three first studio albums.
- The problem remains unanswered based on the provided section. Additional information would be needed to determine the director of "Big Stone Gap" and their base in New York City.

Chunk3

...

Document 81: Big Stone Gap (film)

Big Stone Gap is a 2014 American drama romantic comedy film written and directed by **Adriana Trigiani** and produced by Donna Gigliotti for Altar Identity Studios, a subsidiary of Media Society.

...

Document 96: Adriana Trigiani

Adriana Trigiani is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in **Greenwich Village, New York City**.

...

Memory3

Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn's three first studio albums.
- **Adriana Trigiani** is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in Greenwich Village, New York City. Trigiani has published a novel a year since 2000.
- **Big Stone Gap** is a 2014 American drama romantic comedy film written and directed by Adriana Trigiani and produced by Donna Gigliotti for Altar Identity Studios, a subsidiary of Media Society. Based on Trigiani's 2000 best-selling novel of the same name, the story is set in the actual Virginia town of Big Stone Gap circa 1970s. The film had its world premiere at the Virginia Film Festival on November 6, 2014.

The director of the romantic comedy "Big Stone Gap" is Adriana Trigiani, who is based in **Greenwich Village, New York City**.

This trajectory highlights several core capabilities learned by the memory agent: estimating potentially relevant content from query keywords and storing it preemptively, immediately updating memory upon encountering context that matches the query, and remaining unaffected by irrelevant information. Notably, these memory behaviors are not the result of architectural attention mechanisms, but emerge as text generation abilities reinforced through RL.

5 Conclusion

We propose a novel approach to modeling long-context tasks by introducing a latent variable memory. This enables the decomposition of continuous autoregressive generation process into a series of steps that sequentially generate context from memory. Our method can handle infinitely long input text with $O(N)$ computational complexity, based on existing Dense-Attention Transformers, without altering the generation paradigm or introducing additional model architectures. We introduce MEMAGENT to implement this modeling approach, equipping LLMs with an RL-trained memory, allowing the model to learn the ability to record relevant information and ignore irrelevant details. Experiments show that when trained on 32K-length data with 8K context (including a 1024-token memory and processing 5000 tokens of input per step), the model can extrapolate to 3.5M with almost lossless performance during testing. Ablation studies demonstrate the effectiveness of using the memory itself as a long-context processing mechanism, as well as the benefits of further RL training on top of it. The results on both in-domain and out-of-domain tasks show that MEMAGENT surpasses long-context post-trained models, reasoning models and other baselines, achieving state-of-the-art performance on long-context tasks.

References

- [1] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? [arXiv preprint arXiv:2404.06654](#), 2024.
- [2] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. [arXiv preprint arXiv:1809.09600](#), 2018.
- [3] OpenAI. Learning to reason with llms, 2024.
- [4] Google DeepMind. Gemini 2.0 flash thinking, 2024.
- [5] XAI. Grok 3 beta — the age of reasoning agents, 2024.
- [6] Anthropic. Claude 3.5 sonnet, 2024.
- [7] OpenAI. GPT4 technical report. [arXiv preprint arXiv:2303.08774](#), 2023.
- [8] Anthropic. Introducing claude 4, 2025.
- [9] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. [arXiv preprint arXiv:2501.08313](#), 2025.
- [10] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. [arXiv preprint arXiv:2405.04434](#), 2024.
- [11] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. [Neurocomputing](#), 568:127063, 2024.
- [12] bloc97. NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation., 2023.
- [13] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. [arXiv preprint arXiv:2306.15595](#), 2023.
- [14] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. [arXiv preprint arXiv:2309.00071](#), 2023.
- [15] Chenxin An, Fei Huang, Jun Zhang, Shansan Gong, Xipeng Qiu, Chang Zhou, and Lingpeng Kong. Training-free long-context scaling of large language models. [arXiv preprint arXiv:2402.17463](#), 2024.
- [16] Xiaoran Liu, Hang Yan, Shuo Zhang, Chenxin An, Xipeng Qiu, and Dahua Lin. Scaling laws of rope-based extrapolation. [arXiv preprint arXiv:2310.05209](#), 2023.
- [17] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. [arXiv preprint arXiv:2309.16039](#), 2023.
- [18] Chaochen Gao, Xing Wu, Zijia Lin, Debing Zhang, and Songlin Hu. Nextlong: Toward effective long-context training without long documents. [arXiv preprint arXiv:2501.12766](#), 2025.
- [19] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. [arXiv preprint arXiv:2004.05150](#), 2020.
- [20] Guangxiang Zhao, Junyang Lin, Zhiyuan Zhang, Xuancheng Ren, Qi Su, and Xu Sun. Explicit sparse transformer: Concentrated attention through explicit selection. [arXiv preprint arXiv:1912.11637](#), 2019.
- [21] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. [arXiv preprint arXiv:2309.17453](#), 2023.
- [22] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. [arXiv preprint arXiv:1904.10509](#), 2019.

- [23] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [24] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmllingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.
- [25] Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023.
- [26] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [27] Jiaxin Zhang, Yiqi Wang, Xihong Yang, Siwei Wang, Yu Feng, Yu Shi, Ruichao Ren, En Zhu, and Xinwang Liu. Test-time training on graphs with large language models (llms). In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 2089–2098, 2024.
- [28] George A Miller et al. The magical number seven, plus or minus two. *Psychological review*, 63(2):81–97, 1956.
- [29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [30] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [31] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2014.
- [32] Jie Ouyang, Ruiran Yan, Yucong Luo, Mingyue Cheng, Qi Liu, Zirui Liu, Shuo Yu, and Daoyu Wang. Training powerful llm agents with end-to-end reinforcement learning, 2025.
- [33] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [34] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- [35] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [36] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.
- [37] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- [38] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. RwkV: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.
- [39] Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.
- [40] Leo Feng, Frederick Tung, Hossein Hajimirsadeghi, Mohamed Osama Ahmed, Yoshua Bengio, and Greg Mori. Attention as an rnn. *arXiv preprint arXiv:2405.13956*, 2024.
- [41] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- [42] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. Moba: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189*, 2025.
- [43] Pedro Henrique Martins, Zita Marinho, and André FT Martins. ∞ -former: Infinite memory transformer. *arXiv preprint arXiv:2109.00301*, 2021.

- [44] Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. Memformer: A memory-augmented transformer for sequence modeling. [arXiv preprint arXiv:2010.06891](#), 2020.
- [45] Aydar Bulatov, Yuri Kuratov, Yermek Kapushev, and Mikhail S Burtsev. Scaling transformer to 1m tokens and beyond with rmt. [arXiv preprint arXiv:2304.11062](#), 2023.
- [46] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.
- [47] Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. Memochat: Tuning llms to use memos for consistent long-range open-domain conversation. [arXiv preprint arXiv:2308.08239](#), 2023.
- [48] Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. Ret-llm: Towards a general read-write memory for large language models. [arXiv preprint arXiv:2305.14322](#), 2023.
- [49] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022.
- [50] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. [arXiv preprint arXiv:2212.08073](#), 2022.
- [51] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. [arXiv preprint arXiv:2501.12948](#), 2025.
- [52] Qwen. Qwq-32b: Embracing the power of reinforcement learning, 2024.
- [53] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. [arXiv preprint arXiv:2501.12599](#), 2025.
- [54] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. [arXiv preprint arXiv:1707.06347](#), 2017.
- [55] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018.
- [56] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. [arXiv preprint arXiv:2402.03300](#), 2024.
- [57] Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. [arXiv preprint arXiv:2501.03262](#), 2025.
- [58] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. [arXiv preprint arXiv:2503.20783](#), 2025.
- [59] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning, 2025.
- [60] ByteDance Seed, Jiaze Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wenyuan Xu, et al. Seed1. 5-thinking: Advancing superb reasoning models with reinforcement learning. [arXiv preprint arXiv:2504.13914](#), 2025.
- [61] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. [arXiv preprint arXiv:2409.19256](#), 2024.

- [62] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, Junyang Lin, Kai Dang, Kexin Yang, Le Yu, Mei Li, Minmin Sun, Qin Zhu, Rui Men, Tao He, Weijia Xu, Wenbiao Yin, Wenyuan Yu, Xiafei Qiu, Xingzhang Ren, Xinlong Yang, Yong Li, Zhiying Xu, and Zipeng Zhang. Qwen2.5-1m technical report. [arXiv preprint arXiv:2501.15383](#), 2025.
- [63] Fanqi Wan, Weizhou Shen, Shengyi Liao, Yingcheng Shi, Chenliang Li, Ziyi Yang, Ji Zhang, Fei Huang, Jingren Zhou, and Ming Yan. Qwenlong-1l: Towards long-context large reasoning models with reinforcement learning. [arXiv preprint arXiv:2505.17667](#), 2025.
- [64] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. [arXiv preprint arXiv:2412.15115](#), 2024.
- [65] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. [arXiv preprint arXiv:1606.05250](#), 2016.

Appendix

A Computation Complexity

We adopt the floating-point operations (FLOP) estimator for the Qwen2Model from verl [61] to compute the FLOP cost of both the baseline model and our proposed method. The results are shown in Figure 7. The baseline model exhibits an $O(n^2)$ complexity, while MEMAGENT achieves an $O(n)$ complexity.

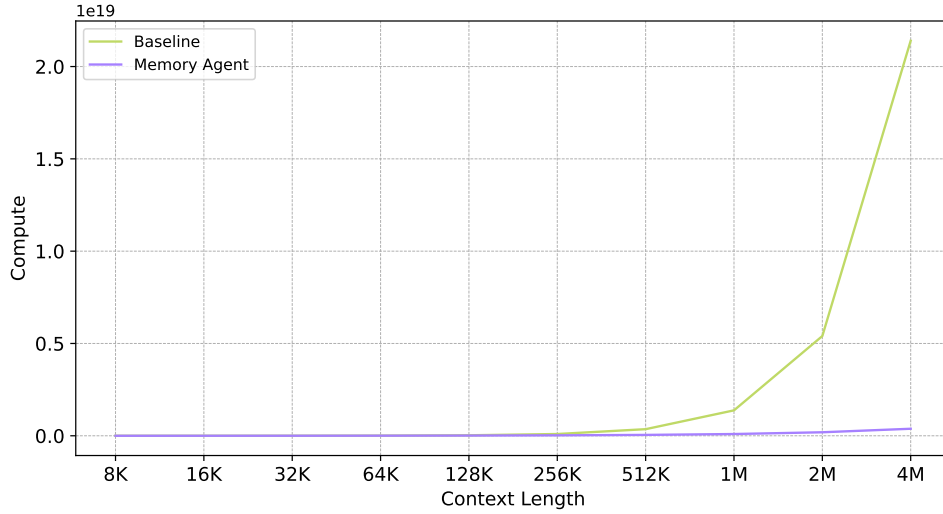


Figure 7 Floating point operations across context lengths from 8K to 4M

For the baseline model, the number of tokens required to process is $q + c + o$, where q represents the length for the problem, c is the context length and o represents the output length.

For MEMAGENT, total FLOP cost is the sum of the FLOPs from all stages. The detailed stages involved are as follows:

- **Initializing:** In the first stage, the model processes an input consisting of $q + 200 + o$, where 200 represents a constant added to prompt the model to follow the MEMAGENT workflow.
- **Memory Updating:** The number of repetitions is determined by $k = \lceil \frac{c}{N} \rceil$, where c is the variable component of the input. Each repetition requires an input of length $q + 200 + N + o$.
- **Final Answering:** The final stage processes an input of length $q + 100 + o$, which includes the accumulated output from the previous steps.

We set $q = 1024$, $o = 1024$, $N = 5000$ and c is ranging from 8K to 4M to calculate the final result.

B Complete Out-Of-Domain Task Results

The RULER benchmark comprises tasks across four primary categories: retrieval, multi-hop tracing, aggregation, and question answering. Each task is designed to evaluate specific aspects of long-context modeling capability through automatically generated examples based on configurable input parameters that define sequence length and complexity. Within this constrained evaluation framework, task complexity can be conceptualized as a function of the number of target output tokens and the signal-to-noise ratio within the context.

The Needle-in-a-Haystack (NIAH) paradigm evaluates retrieval performance in long-context scenarios by inserting key-value pairs ("needles") into large distractor content ("haystack"). It includes four variants: Single NIAH, which requires retrieving a single needle; Multi-keys NIAH, which involves retrieving one needle amidst multiple distractors; Multi-values NIAH, where all values associated with a key must be extracted; and Multi-queries NIAH, which necessitates retrieving multiple needles with distinct keys. Variable Tracking (VT) tests multi-hop reasoning by tracking entity chains across extended sequences, while Frequent Words Extraction (FWE) challenges models to identify frequent words in power-law distributions, evaluating their ability to analyze word frequencies in linguistic data.

B.1 Needle-in-a-Haystack (NIAH)

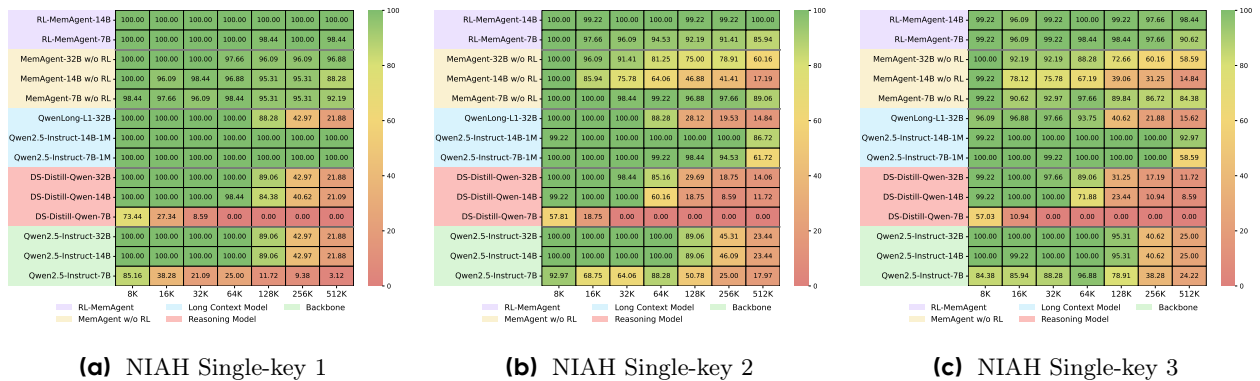


Figure 8 Performance on Single-keys NIAH tasks with increasing numbers of distractor needles. Tasks 1-3 represent different levels of difficulty with varying distractor densities.

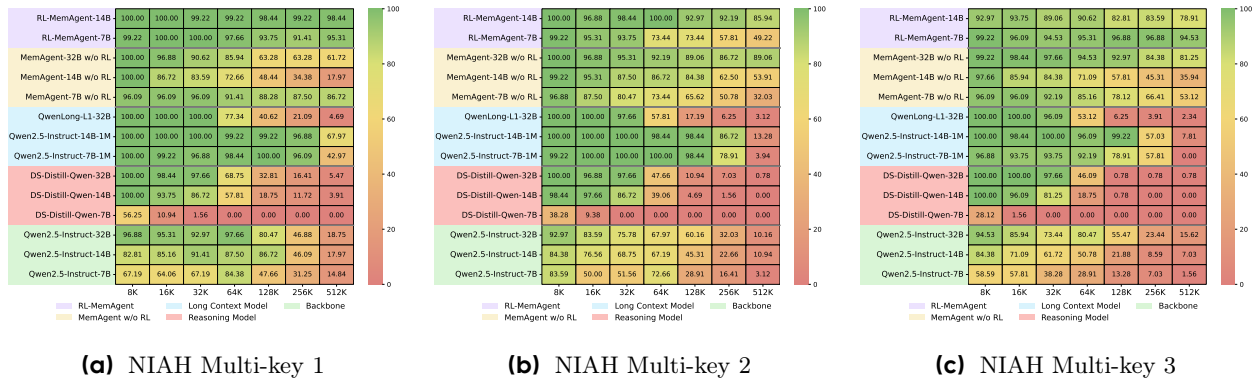
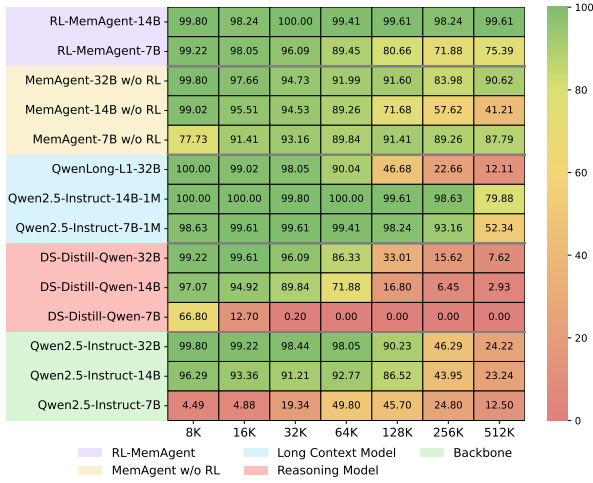
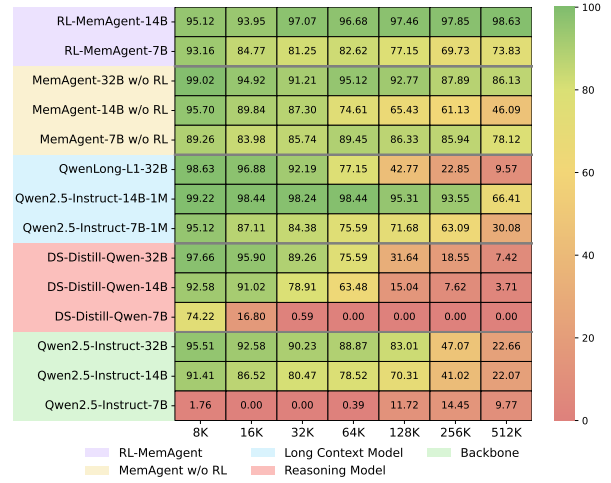


Figure 9 Performance on Multi-keys NIAH tasks with increasing numbers of distractor needles. Tasks 1-3 represent different levels of difficulty with varying distractor densities.



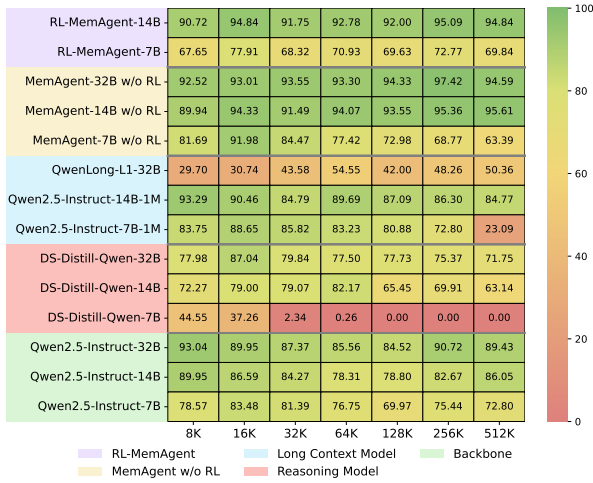
(a) NIAH Multi-query



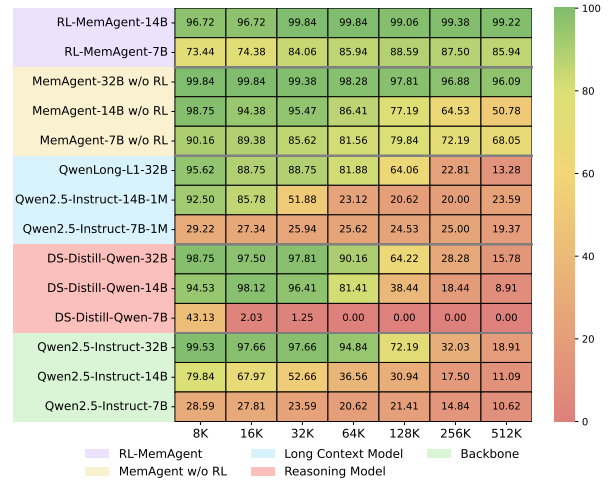
(b) NIAH Multi-value

Figure 10 Performance on advanced NIAH variants. (a) Multi-query task requiring retrieval of multiple distinct needles. (b) Multi-value task requiring extraction of all values sharing identical keys.

B.2 Variable Tracking (VT) and Frequent Words Extraction (FWE)



(a) Frequent Words Extraction



(b) Variable Tracking

Figure 11 Performance on frequent words extraction and variable tracking tasks.